

Technical Evaluation Report

JAVA Assessment

Candidate:	nitin.bharti@irissoftware.com
Email:	nitin.bharti@irissoftware.com
Tech Stack:	JAVA
Completed:	July 29, 2026 at 04:21 PM
Report Generated:	July 29, 2026 at 04:22 PM

Overall Assessment

68.0 / 100

Evaluation Duration

Metric	Value
Actual Time Taken	24.7 minutes
Planned Duration	30 minutes
Time Efficiency	82.3% of planned time ✓

Category Scores

Category	Score
Java 18	100.0/100
Java 17	67.0/100
Java 20	41.5/100
Java 19	97.5/100
Java 21	0.0/100

Communication Assessment

Metric	Score
Confidence Level	7.7/10
English Proficiency	8.7/10
Clarity Score	8.2/10

Strengths

- Correctly identifies the introduction of a new package and unified interface
- Clear explanation of pattern matching enhancement
- Good use of switch statement
- Provided clear examples of how previewing features allows real-time feedback and design changes
- Mentions specific names of new algorithms (LXM, Zoro)

Areas for Improvement

- Could have elaborated more on the safety net concept, explaining how it prevents permanent feature removal without breaking code
- Did not explicitly mention that tooling preparation is also a reason for previewing features
- Candidate did not provide an answer
- Elaborate on what 'superior speed statistical quality and randomness' means in practical terms
- Correctly state the impact of removing outdated features on security

Detailed Question Feedback

Q1: The Simple Web Server introduced in Java 18 is best suited for:

Category: Java 18 | Question Category: FSUpskilling | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

Prototyping, testing, and ad-hoc file serving

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Prototyping, testing, and ad-hoc file serving' is correct.

Q2: What are the features introduced in Java 17?

Candidate's Verbal Answer:

There are multiple enhancement has been done in Java 17. So in terms of language enhancement, they are introduction of seed classes, the pattern matching enhancement and language enhancement. So in the pattern matching, old version repeated instance of checks and manual casting is basically we were doing. So with the pattern matching, no manual casting is required and that will make the code shorter cleaner and more safer. So it works as a reference type and not a reference type and not on the primitive type. Apart from that, we are having the concept of seed class. So when we want to predefined rustic class, I mean in terms of what all classes can extend it and what all basically will not be extended. So in that case, seed class would come into picture and we are using keyword Seeds, No Seeds, Permits. Those are the keywords that we will use in the seed classes. There some enhancement is being done in the calculation also for the floating point semantics. So floating point calculation more predictable and consistent across the platform in Java 17. So calculation will give the same result regardless of the other driven on the x86 or ARM processor. Apart from that, in terms of the performance and memory improvement, in Java 17 vector API is being introduced. Foreign function and memory API are also being introduced and enhanced pseudo random number generator also is introduced in the Java 17. In terms of security and encapsulation, so Java 17 strength and security by removing outdated feature and enforcing encapsulation. So strong encapsulate JDK internals which prevents access to the internal Java classes which were previously implemented using reflection API, deprecated the security manager from the removal and context-specific desolation parameters are also being added and RMI and applied is also deprecated. So on very high level, these are the basic features that is introduced.

Score: 71/100

Good: Good understanding of key features in Java 17, Clear explanation of pattern matching enhancement

Improve: Clarify the concept of seed classes and their usage, Correctly state the impact of removing outdated features on security

Detailed Feedback:

The candidate's answer demonstrates a good understanding of Java 17 features, but with some minor errors and omissions. They correctly identified pattern matching enhancement, seed classes, and floating-point calculation improvements as key features. However, they missed the introduction of vector API, foreign function and memory API, and enhanced pseudo-random number generator. Additionally, their explanation of seed classes was somewhat unclear, and they incorrectly stated that Java 17 removes outdated features to enforce encapsulation (it actually strengthens security by removing outdated features).

Q3: Why does the JDK preview a feature across multiple releases (like virtual threads across 19, 20, and finalized in 21) instead of shipping it once?

Category: Java 20 | Question Category: FSUpskilling | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

So Java is adding the preview features across multiple releases and it will then come up with the final production like code, right? So yes, so the reason for that is, I mean it basically the first point is safety net which basically permanent feature cannot be removed without breaking code. So preview allow design changes before committing forever, right? And the second point for this is real feedback. So developers give the real-time test edges cases in real project using enable preview. Tooling preparation is also giving ID IntelliJ, Eclipse build tool and framework time to add support. Allows related features like record and pattern matching to adapt and mature together. And let's Java deliver working code quickly to get input early rather than waiting years for the mega-review. So these are the different reasons why Java release the preview in across multiple releases before finalizing any feature.

Score: 83/100

Good: Correctly identified two main reasons for previewing features, Provided clear examples of how previewing features allows real-time feedback and design changes

Improve: Could have elaborated more on the safety net concept, explaining how it prevents permanent feature removal without breaking code, Did not explicitly mention that tooling preparation is also a reason for previewing features

Detailed Feedback:

The candidate's answer demonstrates a good understanding of the reasons behind Java's preview feature across multiple releases. They correctly identify two main points: safety net and real feedback. However, their explanation for

the safety net is slightly unclear, and they could have elaborated more on how this allows design changes before committing to a permanent feature.

Q4: Write a program to check if a string has balanced parentheses using a stack.

Category: Java 19 | Question Category: FSUpskilling | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
import java.util.*;

public class ParanthesesChecker {
    public static boolean isBalanced(String s){
        var stack=new ArrayDeque<Character>();
        for(char c: s.toCharArray()){
            if("({[".indexOf(c) !=1)
                stack.push(c);
            else if("})]".indexOf(c) != -1){
                if(stack.isEmpty() || !matches(stack.pop(),c))
                    return false;
            }
        }
        return stack.isEmpty();
    }
    private static boolean matches(char open,char close){
        return switch(open){
            case '(' -> close == ')';
            case '{' -> close == '}';
            case '[' -> close == ']';
            default -> false;
        };
    }
}
```

Score: 95/100

Good: Efficient use of a stack, Good use of switch statement

Improve: Minor issue with indexOf(c) !=1 should be != -1

Detailed Feedback:

The code correctly solves the problem of checking if a string has balanced parentheses using a stack. It uses a stack to keep track of opening brackets and checks for matching closing brackets. The matches function is used to determine if an opening bracket matches with its corresponding closing bracket. However, there's a minor issue in the line 'if("({[".indexOf(c) !=1)', which should be 'if("({[".indexOf(c) != -1)' instead of 1. ■■ Note: Compilation failed due to environment/infrastructure issues (e.g. missing JUnit/Spring jars in the runner). No score penalty applied. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:options to suppress this message. Use -proc:none to disable annotation processing. Error: Main method not found in class ParanthesesChecker, please define the main method as: public static void main(String[] args) or a JavaFX application class must extend javafx.application.Application

Q5: Write a program to implement a trie (prefix tree) supporting insert and search operations.

Category: Java 21 | Question Category: FSUpskilling | Difficulty: medium | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q6: The Vector API in Java 18 is delivered as a(n):

Category: Java 18 | Question Category: FSUpskilling | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

Second incubator module

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Second incubator module' is correct.

Q7: What are enhanced pseudo-random number generators introduced in Java 17 (JEP 356)?

Category: Java 17 | Question Category: FSUpskilling | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

So, so the random number generator introduced in Java 17 with the following it's a basically introduced in there as a new package and unified interface. So, it's moved to the Java.tutle.random all generators legacy and new implemented the random generator interface. Fatnum factory in instantiate algorithm by name via the random editor factory dot off and in bracket we can pass algorithm name dot create instead of that class instantiation introduce LXM algorithm and Zoro zero algorithm providing superior speed statistical quality and the randomness and it's it is it is having better parallelism specialized interface like the spritable random generator and jumpable random generator allow safe known overlipping same generation across multiple region.

Score: 63/100

Good: Correctly identifies the introduction of a new package and unified interface, Mentions specific names of new algorithms (LXM, Zoro)

Improve: Provide more detailed explanation of the benefits of LXM and Zoro algorithms, Elaborate on what 'superior speed statistical quality and randomness' means in practical terms

Detailed Feedback:

The candidate's answer demonstrates a basic understanding of enhanced pseudo-random number generators introduced in Java 17 (JEP 356). They correctly identify the new package and unified interface, as well as the introduction of LXM and Zoro algorithms. However, their explanation is somewhat unclear and lacks specific details about the benefits of these new algorithms. Additionally, they mention 'superior speed statistical quality and randomness' but do not elaborate on what this means in practical terms.

Q8: Write a program to check if two binary trees are structurally identical.

Category: Java 20 | Question Category: FSUpskilling | Difficulty: medium | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q9: Virtual threads were introduced in Java 19 as a:

Category: Java 19 | Question Category: FSUpskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

Preview feature (JEP 425)

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Preview feature (JEP 425)' is correct.

This report was generated by the Evalo.
© 2026 IRIS Software. All rights reserved.